PROOF LANGUAGE DESIGN / ROUND 3

MORAINE

Evidence-Indexed Refinement and Behavioral Typing for Human-Scale Mathematics

Keep the mathematical object fixed.
Let its usable interface grow with the proof.

Prepared for Vladimir Reshetnikov

GPT-6 Astra Pro

6 September 2026

Status. A language and elaboration design, with explicit mathematical semantics, source-based case studies, and an executable reference checker for a small universal length-contract fragment. This is not an implemented end-to-end language. The accompanying ordinary-Lean reference file was not compiler-tested in this session.

Abstract

Lean can already express continuity, injectivity, naturality, inverse laws, convergence, and program behavior. The difficulty is not an insufficient logic: it is that these facts are distributed among hypotheses, structures, coercions, and theorem applications, while the mathematician treats them as part of what an object *is currently known to be*. The second brainstorming round established a sound architecture for certified computation but left this typing interface largely implicit.

This article proposes MORAINE, a refinement-aware elaboration layer over Lean. Its basic judgment assigns an existing term a carrier together with an evidence-bearing row of properties. Refinement never silently changes the term, its operations, its domain, or its branch. Properties can describe an individual value, an entire function, a relation between several objects, or a statement valid only on a set or near a point. Ordinary proof terms justify refinement introduction, subsumption, property propagation, and conversions to bundled mathlib interfaces. Behavioral synthesis must produce evidence of its exact specification; a well-typed candidate, a retained candidate, and a universally correct candidate are different outcomes.

Four source studies drive the proposal: an autonomous-derivative induction and endpoint bookkeeping in ProveIt, and coinduced invariants and a finite tensor universal property in the Fermat repository. The article derives their shorter proof forms without deleting their mathematical obligations. It also compares a conservative elaborator with possible kernel extensions, states a relative soundness theorem for a restricted refinement calculus, and gives a complete coefficient criterion for universal affine length contracts. The accompanying Python implementation passes 32 regression tests, including 25,600 concrete interpreter comparisons; these tests are explicitly distinguished from a Lean formalization. The recommended first implementation adds property-aware binding, scoped evidence, and exact-target synthesis, not a new trusted equality engine.

Reading map

The core proposal is in Sections 1–5. The source-based proof reconstructions are in Section 6. Sections 7 and 9 address synthesis and the type-system question directly. Section 10 specifies the executable fragment and its exact limitations. The implementation and evaluation plan deliberately allows the experiment to conclude that an improved Lean library is sufficient.

Contents

1 The third-round question: can facts become usable types?

1.1 What the previous round has already settled

The two round-two syntheses, *Mathematical Objects, Certified Computation* and *Nine Workbenches*, converge on a valuable boundary: a computation returns an object together with a proof of a precisely stated relationship to the original request [1, 2]. A polynomial presentation, a power-series jet, a numerical enclosure, and a complete solution set have different contracts. A finite observation must not become an exact identity merely because the interface uses the same convenient notation.

I retain that boundary. I also retain fixed semantic targets, scoped guards, proof-producing replay, explicit execution assumptions, and comparison against ordinary Lean equipped with the same libraries and services. These are inherited requirements, not new contributions of Moraine. In particular, the earlier residual-to-jet arguments, the distinction between preservation and reflection, and the warning that finite behavioral tests are not universal proofs should not be presented as discoveries of a third-round proposal.

The reports are also unusually useful about their own limitations. Their agreement is correlated by common source material and a common brief; it is not independent usability evidence. The nine proposals did not implement a complete language. The *syntheses*, however, did subsequently run focused Lean experiments. It would be inaccurate to flatten these two facts into "round two contained no compiled Lean." The second synthesis also identifies repeated use of the same three example files and warns against replacing implementation with another elaborate workspace design [1, 2].

The new question is therefore not "what other CAS commands should the language have?" It is:

> Can the evidence already present in a mathematical development systematically change how an existing term is admitted, applied, composed, and presented, without changing which term it denotes?

1.2 The mismatch is organizational, not logical

Consider a function $f : A \to B$. A mathematician may know that it is linear, injective, continuous, equivariant, and inverse to a particular g. These are not five different functions. Nor do all five facts have the same shape: linearity quantifies over pairs of arguments and scalars; equivariance refers to two actions; being inverse to g is relational; continuity refers to chosen topologies. A useful language must keep the function and these parameters attached to one another.

Lean's existing dependent types, propositions, subtypes, structures, and implicit arguments already represent such information. Propositional proofs are irrelevant for runtime computation, but their presence remains important to static checking. Propositional equality is also distinct from conversion by definitional equality [10, 11, 12]. The proposed extension must not claim expressive powers Lean lacks only superficially.

The problem is instead that a statement of a property does not automatically provide a predictable *typing interface*. The user may need to introduce a particular local instance, build a bundled map, extract its underlying function, identify that function with another spelling, and select a theorem expecting a slightly different bundle. Some of these steps are already well automated; others expose representation rather than mathematical reasoning.

Moraine makes the following distinction explicit:

$$\underbrace{t : A}_{\text{which object and carrier?}} \qquad \underbrace{P_1(t), \ldots, P_r(t)}_{\text{what has been proved about it?}}$$

The left-hand component is stable. The right-hand component can grow as a proof proceeds. A theorem may ask for a property-bearing interface, and the elaborator may supply it using the evidence already attached to that exact term.

1.3 The proposed increment

The proposal has three tightly related additions.

Refinement-aware typing. Source-level classifications have a fixed Lean carrier and an evidence row. Property introduction and weakening are explicit inference rules, even when the user does not write their proof arguments.

Relational and scope-aware contracts. The row can mention an entire function, several anchored objects, or a set/filter on which a fact holds. Typing obligations distinguish equality at a point from equality near it, and a family of arbitrary equivalences from a natural family.

Evidence-directed synthesis. A request for an inhabitant of a carrier is separate from a request for an inhabitant satisfying a behavioral law. The latter must finish with a proof of that law about the selected inhabitant. Candidate filtering and proof construction may cooperate, but they confer different permissions.

These additions form an extension to the *source typing and elaboration system*. The first implementation is a conservative translation into ordinary Lean. Section 9 also considers genuinely new kernel rules; it does not assume that "do not change the kernel" is an answer to every ergonomic problem.

2 Source study: four distinct kinds of proof overhead

The analysis uses pinned snapshots rather than an unspecified moving branch. Table 1 records the repository revisions. The individual source paths are given in the bibliography and the machine-readable manifest shipped with the article. This is a sample of source code, not a build audit of either mathematical repository and not an assessment of the whole Fermat proof.

Table 1: Snapshots inspected for this proposal. Short hashes identify full hashes in the manifest.

Repository	Revision	Material used
Brainstorm	`b052ec011beb`	Both round-two syntheses
ProveIt	`24ce8bd743ea`	Autonomous derivatives; B-process auxiliaries
`fermats-last-theorem`	`aa2d8b34692b`	Coinduced invariants; finite tensor universal property
Leant	`3a40904be8a4`	Behavioral surface, verification, and selection boundary

2.1 The derivative proof is already mathematically organized

`AutonomousIteratedDeriv.lean` proves that if $W' = \varphi \circ W$ on an open set and $G_{n+1} = G_n'\varphi$ at the relevant values, then $W^{(n)} = G_n \circ W$ there [3]. The source already isolates a reusable theorem and uses a straightforward induction. In its successor case, it turns the induction hypothesis on the open set into eventual equality near x, transfers a derivative, and applies the chain rule.

This is not evidence that every Lean proof is bloated. It is a good example of a small but recurring mismatch: the human statement "differentiate the induction hypothesis near x" spans several library-level objects. The useful compression preserves the open-set and neighborhood reasoning rather than pretending it is unnecessary.

2.2 An interval conversion changes the answer by an error term

`GKBProcessAux.lean` crosses from a closed integer interval to a closed natural interval, then to a half-open natural interval used by the exponent-pair interface [4]. The first step is a bijective reindexing with nonnegativity conditions. The second may discard one endpoint. It therefore yields a norm bound with an additive error, not an equality of sums.

The same file proves that a fixed phase shift and phase reversal preserve the norm of a finite exponential sum. These transformations have a different contract from endpoint deletion. A generic instruction "change to the standard representation" would hide precisely the distinction the proof must preserve.

2.3 Coinduced invariants: one map, several admissibility proofs

`S_Rep_nonempty_invariants_coind_equiv.lean` constructs a linear equivalence between the invariants of a coinduced representation and the invariants of its source representation [5]. The mathematical maps are evaluation at the identity and constant extension. The proof establishes that an invariant coinduced function is constant, that evaluation lands in the required invariant subspace, and that constant extension satisfies both coinduction and invariance conditions.

The source then packages these maps, proves the linearity fields, and uses nested subtype extensionality for an inverse law. Here the opportunity is not a stronger solver. It is a property-aware bridge between an ordinary function with proved laws and the bundled linear equivalence expected by mathlib.

2.4 Finite tensor products: naturality is not a type coincidence

`S_Algebra_exists_algHom_equiv_pi.lean` constructs a finite free algebra W representing families of algebra homomorphisms from finitely many algebras H_i [6]. Its conclusion includes equivalences

$$\mathrm{Hom}_{A\text{-alg}}(W, T) \simeq \prod_i \mathrm{Hom}_{A\text{-alg}}(H_i, T)$$

and a separate naturality law under postcomposition in T. The proof uses induction on the finite index type, the base algebra A, and binary tensor products. It also transports the equivalences through products and index changes.

A family of equivalences alone does not contain the naturality law. The additional law is an excellent example of a behavioral constraint on a higher-order, dependent object. It should be part of the requested interface, not inferred from the word "equivalence."

3 A language the mathematician can read

3.1 Controlled mathematical syntax, not unrestricted prose

All Moraine listings below are *proposed syntax*, not executable Lean. The parser gives words such as `linear`, `on`, and `inverse` exact package-defined meanings. Ordinary prose can accompany a proof, but a sentence outside the controlled grammar does not silently become an axiom or a theorem application.

The basic binding form is:

```
let f : A -> B
know f is continuous
know f is injective
use f as ContinuousMap(A, B)
```

The two `know` commands require proofs or produce visible proof obligations. The last command constructs a registered bundled interface using the same underlying function. It is not a search for a different continuous map.

A binder can state the properties at the point of introduction:

```
fix f : A -> B where continuous, injective
```

This binds an arbitrary function together with assumptions about that function. It does not construct a function or assert that one exists without hypotheses. An author wishing to construct a function uses a different command:

```
construct f : List Nat -> List Nat
  require forall xs, length (f xs) = length xs
  prefer structural
```

Here `require` is part of the goal. `prefer` affects search order only. A candidate accepted for the carrier type does not finish the construction until the universally quantified requirement has also been proved.

3.2 An entire proof, with the mathematical obligations still visible

The coinduced-invariants example can be presented as in Listing 1. The names `Coind`, `fixed`, and `linear_equivalence` come from a representation package whose denotations are fixed by the imported environment. The proof is not licensed by the English-looking appearance of the lines. Each `by` selects a theorem schema or an explicitly bounded proof procedure. For example, `invariance at 1` instantiates an action law and applies congruence to evaluation at 1. Section 6.1 gives the mathematical expansion. The system must show that expansion when asked why E is well-defined. The constancy lemma is explicitly universal and established before the two admissibility blocks; no branch-local function or proof is leaked from one block into another.

The result is still `Nonempty` of a linear-equivalence type, matching the original theorem. Replacing it by an executable globally selected equivalence would be a different request. The local maps are constructed in the proof; no elimination of an arbitrary existential proof into executable data is concealed.

Listing 1: Proposed source for the coinduced-invariants equivalence.

```
theorem coinduced_invariants
  fix k : commutative_ring
  fix G : group, H : subgroup G, N : representation k H
  conclude nonempty linear_equivalence
    (fixed G (Coind H N)) (fixed H N)
proof
  let E(f in fixed G (Coind H N)) := f(1)
  let C(n in fixed H N) := fun x : G => n

  have constant_invariants:
    forall f in fixed G (Coind H N), forall x, f(x) = f(1)
    by invariance at 1

  show E maps fixed G (Coind H N) into fixed H N
    by coinduction covariance and constant_invariants

  show C maps fixed H N into fixed G (Coind H N)
    by covariance and invariance of constant functions

  have E and C are linear by pointwise operations
  have E(C(n)) = n for all n in fixed H N by definition
  have C(E(f)) = f for all f in fixed G (Coind H N)
    by constant_invariants
  finish by inverse_linear_maps E C
end
```

3.3 What must remain explicit

The surface can omit a repeated proof that a particular denominator is nonzero. It cannot omit which denominator, which field structure, or which local hypothesis justifies cancellation. It can infer a bundle adapter. It cannot silently select a different topology or algebra structure because that makes a theorem applicable. It can hide a routine extensionality proof. It cannot hide that the proposed two maps have not been proved inverse.

These distinctions suggest three annotation classes:

Class	Examples and policy
Meaning-bearing	Carrier, operations, domains, branches, quantified variables, requested behavior. Fixed before discretionary proof search.
Evidence-bearing	Nonzero proof, continuity proof, equality transport, linearity law. May be synthesized, but must be checked and scoped.
Search-only	Preferred lemma, provider order, work budget, display verbosity. May change which proof is found, not the declared meaning.

Some information is ambiguous until elaboration resolves it. In that case the system either uses an explicit deterministic language rule or requests a meaning-bearing annotation. A proof search success is not a valid rule for choosing between two inequivalent readings of the theorem statement.

3.4 Reason, hint, and open search are different commands

`by inverse_linear_maps E C` commits to that construction and its law obligations. `hint inverse_linear_maps` merely favors that lemma. `solve` permits a broader search within the active profile. These commands may end in extensionally equal proof terms, but they make different claims about the author's stated argument.

This is not a restriction imposed by Lean's logic: proof irrelevance does not require proofs to remember their exposition. It is an explanation contract of the source language. An unused asserted fact must still be proved; an unrelated successful proof of the final theorem does not retroactively validate a false intermediate claim.

4 A small evidence-indexed refinement calculus

4.1 Carrier types and refinement classifications

A first implementation should not introduce a second general-purpose universe hierarchy. Let A be an ordinary Lean type. A *classification over* A is a predicate $P : A \to \mathsf{Prop}$. A finite row of properties is interpreted as their conjunction:

$$\rho = \{P_1, \dots, P_r\}, \qquad [\![\rho]\!](t) = P_1(t) \wedge \cdots \wedge P_r(t).$$

The empty row denotes `True`. A row label is an indexing aid, not a new logical connective. Two labels that elaborate to different predicates are not interchangeable because their names look similar.

The notation

$$A \triangleright \rho$$

means a classification with carrier A. It is deliberately *not* a rule that A has become definitionally equal to a subtype. Refinement intersection is only intersection of properties over an already fixed carrier:

$$A \triangleright P \;\&\; A \triangleright Q = A \triangleright P \wedge Q.$$

There is no implicit intersection of unrelated carrier types.

The underlying mathematical predicates remain ordinary Lean propositions. For a function, the row might contain

$$\mathsf{Continuous}_{\tau_A,\tau_B}(f), \quad \mathsf{Injective}(f),$$
$$\mathsf{Linear}_{k,M,N}(f), \quad \forall x,\ g(f(x)) = x.$$

The indicated topologies, modules, scalar ring, and second function are semantic parameters. The implementation may store them in shared expression nodes, but must not forget them when indexing a property.

4.2 The judgment and its output

Write

$$\Gamma; \Delta \vdash e \Rightarrow A \triangleright \rho \rightsquigarrow (t; \pi) \tag{J}$$

for elaboration of source expression e. Here Γ is the telescope of carrier-level variables and fixed structure parameters; Δ is the current evidence context; t is a Lean expression of type A; and π

is a collection of Lean proofs of $[\![\rho]\!](t)$. For a checking judgment, an expected carrier and row are supplied rather than inferred.

An implementation need not build a large conjunction at every use. It may store one checked proof per property and build a conjunction only at an interface boundary. The invariant is semantic: all stored proofs concern the exact term t under the exact context in which they are used.

There are two important kinds of identifier. An object handle points to the elaborated value expression, with universe and instance arguments resolved. An evidence handle points to a proof and its scope. Neither an identifier spelling nor a pretty-printed string is a sufficient identity for either handle.

4.3 Evidence introduction, weakening, and transport

The following rules describe the initial core. Each premise involving evidence must ultimately produce an ordinary Lean proof.

Introduction. If $t : A$ and $p : P(t)$, then t has classification $A \triangleright P$ with evidence p. The carrier-level term remains t.

$$\frac{\Gamma; \Delta \vdash t : A \qquad \Gamma; \Delta \vdash p : P(t)}{\Gamma; \Delta \vdash t : A \triangleright P}. \tag{R-Intro}$$

Conjunction and projection. Evidence of $P(t)$ and $Q(t)$ produces evidence of $(P \wedge Q)(t)$; evidence of a conjunction can be projected. These are applications of the corresponding Lean constructors and eliminators, not an independent trusted property database.

Subsumption. A classification may be weakened by an implication:

$$\frac{t : A \triangleright P \qquad h : \forall x : A,\ P(x) \to Q(x)}{t : A \triangleright Q}. \tag{R-Sub}$$

It is often cheaper to prove the specialized implication $P(t) \to Q(t)$ instead; the implementation may use that version. The displayed rule explains reusable interface inclusion. The word "subtyping" here denotes proof-mediated subsumption, not kernel conversion.

Equality transport. If $p : P(t)$ and $h : t = u$, the system can construct a proof of $P(u)$ by the equality eliminator. It cannot simply change the object handle of p. If t and u are related only by an equivalence, an approximation, or a map that preserves some properties, the corresponding transport theorem must be supplied instead.

Application of a behavioral contract. For a total function $f : A \to B$, define

$$\mathsf{Contract}(P, Q, f) := \forall x : A,\ P(x) \to Q(x, f(x)). \tag{1}$$

Here $Q : A \to B \to \mathsf{Prop}$ may relate input and output. A proof of $\mathsf{Contract}(P, Q, f)$ and a proof of $P(a)$ produce $Q(a, f(a))$. Application returns *the original* $f(a)$, not a newly synthesized b for which $Q(a, b)$ happens to hold.

4.4 Contract variance is an implication theorem

Suppose f satisfies $\mathsf{Contract}(P_1, Q_1, f)$. It also satisfies $\mathsf{Contract}(P_2, Q_2, f)$ if

$$\forall x,\ P_2(x) \to P_1(x), \qquad \forall x\, y,\ P_2(x) \to Q_1(x, y) \to Q_2(x, y). \tag{2}$$

The precondition is weakened in the direction required by the caller: every input admitted by the requested interface must be admitted by the implementation. The output obligation can be weakened only by a proved implication. The proof is immediate by applying the original contract and then the second implication.

This matters for familiar mathematical properties. A theorem about a function on all of U can be used on $V \subseteq U$. A theorem known only on V cannot be promoted to U because V was the most recently mentioned set. Likewise, Lipschitz constant L can be enlarged to $M \geq L$, but a smaller constant requires additional evidence.

4.5 A compositional rule with relational postconditions

Let $f : A \to B$ and $g : B \to C$. Suppose

$$\begin{aligned} &h_f : \forall x,\ P(x) \to Q(x, f(x)),\\ &h_g : \forall y,\ R(y) \to S(y, g(y)),\\ &h_{\text{link}} : \forall x\, y,\ P(x) \to Q(x, y) \to R(y). \end{aligned}$$

Then

$$\forall x,\ P(x) \to \exists y,\ Q(x, y) \wedge S(y, (g \circ f)(x)). \tag{3}$$

Use the witness $y = f(x)$, apply h_f, derive $R(f(x))$ with h_{link}, and apply h_g. If an application needs a more convenient postcondition $T(x, z)$, it can provide a theorem $Q(x, y) \wedge S(y, z) \to T(x, z)$.

This is ordinary compositional specification, close in spirit to pre/postcondition systems [19]. Its role in Moraine is to make the law a typing interface for mathematical composition. For example, composition of injective maps or monotone maps is not guessed from examples of inputs; it is licensed by an appropriate theorem schema.

4.6 Local flattening versus first-class packaging

A subtle implementation distinction prevents "no wrappers" from becoming a false promise. In a local binder

```
fix x : A where P(x)
```

the compiler can introduce `x : A` and a separate proof parameter `hx : P x`. The source keeps one mathematical name, while the core telescope contains the necessary evidence.

When such a refined object must be stored in a list, returned from a function, or passed to a function whose argument is itself a refined package, evidence must cross a first-class boundary. A conventional encoding is:

```
structure Certified (A : Type u) (P : A -> Prop) where
  value : A
  property : P value
```

This is essentially a subtype. The proposed benefit is not that subtypes cease to exist. It is that they are inserted at actual boundaries rather than becoming the visible identity of every local object. The elaborator maintains equations between projections and the original anchored terms; consumers cannot assume a wrapper is definitionally absent in every dependent context.

There is a second distinction. The classification

$$(A \to B) \triangleright \mathsf{Contract}(P, Q, -)$$

contains a *total* function on A, with a guarantee on inputs satisfying P. By contrast, a function

$$\{x : A \mid P(x)\} \to B$$

has a restricted input type. The two are not interchangeable. A local proof-argument translation of the latter has the shape $\forall x : A,\ P(x) \to B$, not $A \to B$. Erasure must not accidentally grant a restricted-domain function an implementation outside its domain.

The initial calculus therefore keeps ordinary Lean dependent types intact and flattens only documented binder forms. An arbitrary family depending on a packaged subtype value is passed an actual package. Proof irrelevance is not a license to erase type dependencies or to fabricate inhabitants of empty refinements.

4.7 No general principal refinement is promised

The inferred row is a finite, demand-driven summary, not the strongest predicate true of the term. For any proposition Q, the question whether $t : A$ can be classified by the constant predicate $\lambda\_.Q$ contains the question whether Q can be proved. A system deciding all such entailments would be a general theorem decider. The practical design is an intentionally incomplete elaborator whose successful answers carry evidence.

Properties may be suggested automatically, but making a property available for checking requires its proof. A timeout means that the current method did not establish the property. It does not mean the property is false or that the underlying object is ill-defined.

4.8 Relative soundness of the restricted core

Theorem 4.1 (Evidence elaboration soundness)**.** *Consider the fragment consisting of ordinary Lean terms, refinement introduction, conjunction/projection, proof-mediated subsumption, equality transport, contract application/composition, and explicit context abstraction and case analysis using only eliminations admitted by Lean for the motive's sort. Suppose its primitive property and transport rules are registered with well-typed Lean proof terms in a fixed environment. Every completed derivation of (J), with no unassigned obligations, translates to a Lean term $t : A$ and Lean proofs of every property in ρ about that term.*

Proof. Induct on the elaboration derivation. An ordinary term is checked by the backend. Refinement introduction retains that term and the supplied proof. Conjunction and projection translate to `And.intro`, `And.left`, and `And.right`. Subsumption applies the implication proof to the existing property evidence. Equality transport uses the equality eliminator with the predicate as its motive. Contract application is dependent function application to the argument and its precondition proof. Composition is the explicit witness construction in (3).

Context abstraction translates to a lambda over the corresponding variable or proof binder. Context extension preserves the earlier telescope and makes a proof available only below that binder. Case analysis translates to the appropriate eliminator, with the common conclusion proved under every branch assumption. Thus a branch-local assumption is either discharged or remains a parameter of an exported result; it does not become an unconditional theorem. Each step is a Lean term-forming operation, so the induction yields the claimed terms and their types. Final backend checking validates the resulting expressions rather than trusting the elaborator's account of this induction. □

This is a metatheoretic argument for the specified fragment, not a mechanized proof of a production parser or elaborator. It does not prove that arbitrary mathematical prose was interpreted as its author intended. It also does not justify an implementation that replaces a registered theorem with a Boolean callback. Those are separate obligations.

Corollary 4.2 (No new theorem axioms are required by refinement itself)**.** *For the fragment of Theorem 4.1, the additional source classifications require no axioms beyond those used by their translated proof terms and imported environment.*

Proof. Every refinement rule expands into the stated existing term constructors or a registered theorem application. None introduces a new constant declared to be a proof without a body. □

5 Scopes, neighborhoods, and laws between objects

5.1 Evidence has a telescope, not just a truth flag

A useful evidence entry has the conceptual form

$$(\text{anchors},\ P,\ p : P,\ \text{binder dependencies},\ \text{environment},\ \text{origin}). \tag{4}$$

The dependencies are the free variables of the actual proposition and proof, closed transitively through local definitions and other evidence. The origin is for explanation and debugging, not logical authority. A receipt saying that a provider "verified" something is not a replacement for $p : P$.

Scopes should be persistent contexts: a child extends its parent without mutating the parent's evidence. A proof discovered in one branch cannot be looked up in a sibling merely because its pretty-printed conclusion matches. At a join, the system may keep a common consequence only by constructing a proof from the case split. It may also export a conditional fact such as $H \to P$ by abstracting over H.

An implementation does not need a second stateful theorem universe to achieve this. Lean already has local contexts and proof abstraction. What is new is the source-level association between those contexts and the advertised refinement rows, plus an auditable account of the association.

5.2 Point, set, and neighborhood modalities

For predicates $P : X \to \mathsf{Prop}$, define

$$\mathsf{At}_x P := P(x), \qquad \mathsf{On}_U P := \forall y \in U,\ P(y), \qquad \mathsf{Near}_x P := \forall^{\text{eventually}} y \in \mathcal{N}(x),\ P(y). \tag{5}$$

These are three propositions, not three display decorations. If U is open and $x \in U$, then

$$\mathsf{On}_U P \longrightarrow \mathsf{Near}_x P. \tag{6}$$

Indeed $U \in \mathcal{N}(x)$; combine this neighborhood membership with the pointwise proof of P on U. There is no corresponding rule $\mathsf{At}_x P \to \mathsf{Near}_x P$.

For equality of functions, this distinction is decisive. The functions $f(t) = t$ and $g(t) = 0$ agree at 0 but have different derivatives there. In a `near x` proof block, introducing a symbol y means reasoning about the predicate under the neighborhood filter; it must not let the author conclude a fact about every y or substitute an arbitrary exceptional y into an eventual statement.

The first implementation should support explicit set and filter propositions rather than a new general modal kernel. A surface scope packages the relevant quantifiers and a small collection of certified conversion theorems. Operations such as differentiating an equality ask for the necessary modality. If only pointwise equality is available, the diagnostic names that mismatch rather than trying every derivative lemma in the library.

5.3 Two-input behavior cannot always be reduced to one-input output facts

Many mathematical adjectives are laws of the form

$$\mathsf{Rel}(R, S, f) := \forall x\, x',\ R(x, x') \to S(f(x), f(x')). \tag{7}$$

Monotonicity uses order for R and S. An isometry uses equality of distances. Injectivity has the reverse implication, or can be expressed using a relation that rules out equal images of distinct inputs. These properties can be stored as predicates of the entire function; the calculus does not force them into a poorly fitting output-only annotation.

Other laws involve several object anchors:

$$\begin{aligned} \mathsf{InversePair}(f, g) &:= (\forall x,\ g(f(x)) = x) \wedge (\forall y,\ f(g(y)) = y), \\ \mathsf{Equivariant}(f, \alpha, \beta) &:= \forall h\, x,\ f(\alpha(h, x)) = \beta(h, f(x)). \end{aligned} \tag{8}$$

Attaching an inverse law only to the type $A \to B$, without the particular f and g, would be an obvious indexing error. More subtle errors occur when two module structures on the same carrier are mistaken for one another. Every structure argument that affects (8) participates in the anchor.

5.4 Naturality is a refinement of a family, not a search heuristic

Let F and G be functors with a fixed action on objects and morphisms. A family $e_T : F(T) \simeq G(T)$ is naturally compatible when

$$G(u) \circ e_T = e_{T'} \circ F(u) \qquad (u : T \to T'). \tag{9}$$

The carrier contains the family of equivalences. Equation (9) is the refinement. Changing the family to one obtained by arbitrary independent choices at each T may preserve each individual equivalence type and destroy the law.

A package may expose `natural equivalence` as a classification or a familiar bundled object. The important feature is that synthesis of the family also creates the naturality obligation, and exporting it as a bundled natural isomorphism supplies that proof. The source does not equate "isomorphism at every object" with "natural isomorphism."

5.5 Operations and their laws must not be confused

A commutative-ring structure contains operations. An assumption that a particular map is injective is a proposition. A basis contains chosen vectors; finite-dimensionality is a property. A selected inverse algorithm contains data; existence of an inverse is a proposition. These distinctions determine what can be hidden by proof irrelevance and what constitutes a mathematical choice.

Moraine keeps Lean's typeclass mechanism for structural data and established interfaces. It does not turn every discovered property into a global instance. A separate scoped index supplies ordinary proof arguments to candidate theorem applications and constructs temporary bundles when their registered adapters apply. A candidate requiring a different operation dictionary is not a proof-only alternative to the current candidate.

5.6 Coherence is local and evidence-based

Suppose two routes bundle the same f as a linear map. If the target structures have the same operation parameters and their underlying maps are definitionally the same, proof irrelevance can dispose of proof-field differences where the structure permits. If the underlying terms are merely propositionally equal, the adapter must use an extensionality theorem. If they differ as functions, the routes are not interchangeable even if both satisfy the advertised property.

The language need not solve global coherence of all possible representation routes. It must select one route, retain it, and prove the property required by the consumer. A request that explicitly compares two routes creates a separate coherence obligation. This avoids both an unrealistic global-confluence promise and the unsafe assumption that every route through an "equivalent" object is judgmentally invisible.

6 Worked reconstructions of the source proofs

6.1 Coinduced invariants and behavioral bundling

Fix the commutative scalar ring, group, subgroup, and representation parameters of the source theorem. In the function presentation used by the proof, a coinduced element $f : G \to N$ satisfies

$$f(hx) = \rho(h)(f(x)) \qquad (h \in H,\ x \in G), \tag{10}$$

and the G-action is right translation:

$$(g \cdot f)(x) = f(xg). \tag{11}$$

These are the conventions of the inspected source; other coinduction conventions would require different formulas [5].

If f is G-invariant, substitute $x = 1$ in (11) to obtain $f(g) = f(1)$ for every g. Substituting $x = 1$ in (10) now gives

$$\rho(h)(f(1)) = f(h) = f(1),$$

so $E(f) = f(1)$ is H-invariant. Conversely, if n is H-invariant, the constant function $C(n)(x) = n$ satisfies (10) and (11), hence defines a G-invariant coinduced element. Pointwise operations give

$$E(f + f') = E(f) + E(f'), \quad E(af) = aE(f)$$

and the analogous equations for C. Finally $E(C(n)) = n$, while constancy gives $C(E(f)) = f$. This proves the desired linear equivalence.

The refinement layer exposes the intermediate classifications:

$$\begin{aligned} f &: (G \to N) \triangleright \{\text{coinduction covariance},\ G\text{-invariance}\}, \\ f(1) &: N \triangleright \{H\text{-invariance}\}, \\ E, C &: \text{their fixed function carriers} \triangleright \{\text{linearity},\ \text{the two inverse laws}\}. \end{aligned}$$

These rows must not be implemented by inventing an unbundled representation API parallel to the entire library. Adapters unbundle the existing objects locally, retain the projection equalities, and repackage E and C with the proved laws. The source's nested `Subtype.ext` steps are generated by those adapters.

The construction still requires the crucial idea that invariant coinduced functions are constant. Automatic refinement inference is not supposed to invent the whole representation-theoretic argument. It removes recurring well-definedness and bundling work *after the argument has been stated*.

A negative test should replace $C(n)$ by a nonconstant function with the correct carrier and ask for the same inverse pair. Carrier checking may succeed, but the invariance or left-inverse obligation must remain unsolved. Another should swap one module action while leaving the carrier type unchanged. The property anchor must then prevent a linearity proof for the old action from being reused.

6.2 A representing algebra with natural behavior

For a finite family of commutative A-algebras H_i, define the object-valued specification

$$\mathcal{F}(T) = \prod_i \mathrm{Hom}_{A\text{-alg}}(H_i, T).$$

The source asks for a commutative A-algebra W, finite and free as an A-module, together with a natural family

$$e_T : \mathrm{Hom}_{A\text{-alg}}(W, T) \simeq \mathcal{F}(T) \tag{12}$$

for commutative A-algebra targets T [6]. Its behavioral equation is

$$e_{T'}(u \circ f)(i) = u \circ (e_T(f)(i)). \tag{13}$$

All quantifiers and structure parameters in (13) belong to the specification.

The source proof suggests the following reusable interface:

```
interface Represents(W, F)
  equivalence e(T) : AlgHom(W, T) <-> F(T)
  law naturality(u, f) : e(T')(u after f) = F(u)(e(T)(f))
end
```

Listing 2 gives the induction skeleton. This is concise only when the displayed interfaces and theorems have already been implemented. The same packages must be available to the ordinary-Lean baseline in an evaluation.

For clarity, the binary mathematical step is not a black box. An algebra map $q : H_1 \otimes_A H_2 \to T$ restricts along the two canonical inclusions to q_1, q_2. Conversely, two such maps determine a map with

$$x \otimes y \longmapsto q_1(x)q_2(y).$$

Listing 2: A finite representing-family construction.

```
construct W representing (T => family i, AlgHom(H(i), T))
  where module_finite A W, module_free A W
by finite_index_induction
  empty:
    use A, representing the empty family
  option:
    use tensor W_previous H(new)
    combine the two representing interfaces by tensor_universal_property
    preserve finiteness and freeness by the tensor module theorems
end
```

Commutativity of T supplies the required commutation of the images. The restrictions are inverse to this construction: equality on pure tensors and the tensor-product generation principle prove uniqueness. Postcomposition by $u : T \to T'$ respects this formula because u preserves multiplication, proving naturality. Induction assembles the finite-family case, with $W = A$ for the empty family.

Thus the refinement `natural` is discharged compositionally by existing universal-property theorems. It is not guessed after selecting arbitrary bijections between the sets in (12). A noncommutative target is also not accepted merely because the same product expression parses: its images must commute, or the theorem's domain must remain the original commutative category.

The statement is existential in W and its structures. Moraine must preserve that quantifier structure. A command that globally chooses a canonical W and exposes executable operations may be useful, but it is a separate construction and cannot be obtained by silently eliminating arbitrary existence evidence.

6.3 Autonomous derivatives and the scope of induction

Let $s \subseteq \mathbb{R}$ be open, and suppose, exactly as in the source,

$$\begin{aligned} &W'(x) = \varphi(W(x)) && (x \in s),\\ &G_0(W(x)) = W(x) && (x \in s),\\ &G_n \text{ has derivative } G_n'(W(x)) \text{ at } W(x) && (n \in \mathbb{N},\ x \in s),\\ &G_{n+1}(W(x)) = G_n'(W(x))\varphi(W(x)) && (n \in \mathbb{N},\ x \in s). \end{aligned} \tag{14}$$

The first line means a derivative assertion, not merely an equation involving Lean's total derivative operation. We seek

$$W^{(n)}(x) = G_n(W(x)) \qquad (n \in \mathbb{N},\ x \in s). \tag{15}$$

The source uses the corresponding `HasDerivAt` hypotheses, and requires properties of G_n only at values $W(x)$ with $x \in s$ [3].

Listing 3 shows the neighborhood-sensitive induction. In the successor step the induction hypothesis holds for *all* points of s. Openness turns it into eventual equality near the chosen x. Its derivative therefore agrees with that of $G_n \circ W$. The chain rule yields $G_n'(W(x))\varphi(W(x))$, and the recurrence identifies this with $G_{n+1}(W(x))$.

The refinement-aware elaborator sees a demand for neighborhood equality when it encounters `differentiate`. It finds the on-set equality, the openness proof, and the membership proof, then

Listing 3: Differentiation of an on-set induction hypothesis.

```
show forall n, on s: D^n W = G(n) after W
by induction n
  zero: by the initial identity
  successor n:
    fix x in s
    near x, have D^n W = G(n) after W by openness and induction
    differentiate this equality at x
    apply chain_rule using the derivative contracts in (14)
    conclude by the recurrence in (14)
end
```

applies (6). It must not weaken the induction statement to equality only at a fixed x and then pretend the successor step still works.

No global differentiability of G_n is added. Such a strengthening could make the interface convenient while excluding the Lambert-function applications the source carefully admits. Preserving hypotheses is a substantive correctness condition of a shorter proof, not an aesthetic preference.

6.4 Endpoint deletion as a quantitative relational type

The B-process auxiliary file provides a useful counterweight to examples where all bookkeeping preserves equality. The following elementary theorem captures its finite-sum step in a reusable form.

Proposition 6.1 (Deletion budget)**.** *Let $T \subseteq S$ be finite sets, $g : S \to \mathbb{C}$ be understood as the restriction of an ambient function, $k \in \mathbb{N}$, and $R \geq 0$. Suppose $|S \setminus T| \leq k$ and $|g(i)| \leq R$ for $i \in S \setminus T$. Then*

$$\left|\sum_{i\in S} g(i) - \sum_{i\in T} g(i)\right| \leq kR, \qquad \left|\sum_{i\in S} g(i)\right| \leq \left|\sum_{i\in T} g(i)\right| + kR. \tag{16}$$

Proof. The disjoint decomposition $S = (S\setminus T)\sqcup T$ gives the difference of the two sums as $\sum_{i\in S\setminus T} g(i)$. The triangle inequality bounds its norm by $\sum_{i\in S\setminus T} |g(i)|$, which is at most $|S \setminus T|R \leq kR$. Applying the triangle inequality once more proves the second assertion. □

The source establishes that its half-open natural range is contained in its closed natural range and that the difference has cardinality at most one. Taking $k = 1$ in (16) gives its general R-bound. Unit-norm exponential summands then give the additive error 1 [4]. The natural-range result itself does not require the real lower endpoint to be nonnegative; those hypotheses belong to the preceding integer-to-natural reindexing theorem.

The language can advertise a quantitative relation

$$\mathsf{Close}_\varepsilon(a, b) := |a - b| \leq \varepsilon.$$

Endpoint deletion produces Close_{kR} for the two fixed sums, while bijective reindexing produces equality and a phase change produces equality *of norms*. These three contracts have distinct carriers or predicates. For $S = \{0\}$, $T = \varnothing$, and $g(0) = 1$, the deletion bound is valid with $\varepsilon = 1$ but equality of the sums is false.

The preceding integer-to-natural conversion also retains its obligations. For example, converting the integer interval $\{-1, 0\}$ using `Int.toNat` identifies two indices. With constant summand 1, the

integer sum is 2, not the one-term natural sum. A proof of nonnegativity is not merely a cast hint: it makes the proposed inverse maps actually inverse.

A concise proof can therefore say:

```
reindex the closed integer sum along toNat and integer_cast
  using nonnegative lower endpoint and ordered endpoints
compare with the half-open natural sum
  by deleting at most one term of norm at most 1
conclude the norm bound with additive error 1
```

The first line elaborates to a bijection proof, the second to (16). No automatic "normal form" may replace both by the same equality rewrite.

7 Behavior-directed synthesis and the existing Leant boundary

7.1 What Leant already does

The inspected Leant snapshot already has a concise host-shaped behavioral surface [7]:

```
:synth --where List.length result = List.length arg0 -- List Nat -> List Nat
```

This clause is not executed as arbitrary Lean code. A bounded parser lowers it to the existing Length-contract representation. Target-derived defaults identify eligible list inputs and scalar or pair outputs. The documented runtime uses a command-local assessment context and exact candidate-specific counterexample replay; raw `sat`, `unsat`, and `unknown` statuses do not by themselves authorize rejection. Unsupported cases are reported rather than silently switched to unconstrained synthesis.

The actual `Verification.hs` is equally explicit about its authority. Its opaque `Verified candidate` records acceptance by a supplied callback. The comments specifically distinguish that from behavioral evidence, a solver certificate, or a kernel proof [8]. The behavioral-selection implementation uses a generative epoch to bind decisions to exact candidate occurrences and seals a total partition of the callback batch; its retained wrapper records the association, not the truth of arbitrary behavior claims [9].

These are strengths to preserve. The proposal is not to add another permissive `--where` parser. It is to connect the existing explicit authority boundaries to a property-aware Lean elaboration interface.

7.2 Three different goals must be represented separately

A synthesis request can ask for:

$$\begin{aligned} &\text{carrier inhabitance:} && t : A, \\ &\text{behavioral witness:} && (t : A) \text{ together with } p : P(t), \\ &\text{proof obligation:} && p : Q \text{ for an already fixed proposition } Q. \end{aligned} \tag{17}$$

The distinction is especially important when A is a function type. An inhabitant of $\mathsf{List}(\mathbb{N}) \to \mathsf{List}(\mathbb{N})$ might return the empty list, reverse its input, increment every element, or duplicate the input. Carrier checking alone says nothing about a requested length law.

Even length preservation does not uniquely determine a function. Identity, reversal, and elementwise successor all preserve length. A synthesis command with an explicit data hole may legitimately choose any of them under that sole requirement. An already fixed definition of identity may not be replaced by reversal because its current consumers happen to inspect only length. This is why a behavioral contract refines a term; it does not identify all terms satisfying the same contract.

7.3 An evidence-producing interface

A new provider interface should return a candidate expression and optional proof-producing artifacts under a sealed request. Schematically:

```
Request:
  fixed environment and local telescope
  original carrier or proposition
  explicit data holes and proof holes
  required property expressions
  permitted axioms and work budget

Candidate:
  proposed assignments to authorized data holes
  candidate syntax or reconstructible expression
  proof terms / certificates / residual obligations
  explanation and exact source-edit proposal
```

The receiving Lean process, not the provider, owns the meaning of the target. It checks the candidate's type, checks each property proof at that candidate, and reconstructs the final declaration. A certificate is admitted only through a theorem whose conclusion matches the required proposition. Every remaining metavariable is either solved or reported as an outstanding obligation.

For local proof search, the target is fixed from the beginning. For data synthesis, the carrier and behavioral predicate are fixed, and only explicitly authorized data metavariables may be assigned. Once a candidate is selected, its properties are checked about that candidate. The general policy is not "freeze all metavariables"; it is "classify which metavariables are allowed to choose mathematical data, and do not confuse them with proof search."

7.4 Refinement information can guide search before the final check

Existing carrier-directed synthesis can be improved by demand propagation. If a candidate theorem expects an injective map, the search can first look for a known injectivity refinement rather than building arbitrary bundles. If a composition must be continuous on a set, it can use a composition theorem to request the necessary image and continuity facts. The obligations are theorem premises, not Boolean feature tests.

A useful initial strategy is bidirectional. The expected refinement selects a small family of theorem schemas. The candidate syntax determines the operation at its head. Their intersection determines which subproperties to request. This is close to how mathlib's `fun_prop` already decomposes function properties [16]; Moraine should reuse that infrastructure, not claim it as a new proof procedure. The additional responsibilities are stable object anchors, uniform binder/interface elaboration, and retained evidence usable by subsequent source constructs.

The system can use a counterexample bank to avoid repeating failed candidates. For universal rejection, a counterexample must be checked against the candidate's actual denotation or a sound

abstraction whose relationship to that denotation is established. Unsupported abstraction, resource exhaustion, and lack of a proof are not counterexamples.

7.5 From observational filters to theorem evidence

There are three legitimate routes to a behavioral refinement:

Direct theorem composition. Reconstruct a proof from laws of the candidate's constructors and providers. Length preservation of a composition of length-preserving operations is a simple example.

Certified decision procedure. Translate the candidate and its contract into a formal domain, prove or reuse the translation theorem, and replay a certificate using a sound checker. Section 10 gives a small complete affine fragment.

General proof search. Ask Lean tactics, a synthesis engine, or another untrusted search procedure for a proof term of the exact behavioral statement. The term is then checked normally.

Finite tests can be useful on every route as a rejection heuristic, provided their interpretation is exact enough for the claimed rejection. Passing them alone supplies no universal refinement. A special finite characterization can justify promotion only when its completeness theorem and applicability premises are themselves available; the affine basis theorem below illustrates this exception precisely.

7.6 Classical existence and executable synthesis

A proof of $\exists t : A,\ P(t)$ lives in `Prop`. It can be eliminated while proving another proposition. Producing data in `Type` from arbitrary such evidence requires a legitimate construction or an explicitly accepted choice principle; Lean's restriction on elimination from propositions is relevant here [11].

The surface therefore separates three commands:

```
obtain t with P(t) from h       -- local existential elimination
construct t with P(t)          -- explicit witness plus proof
choose t with P(t) from h      -- declared classical choice policy
```

The exact syntax can change, but the semantic distinction cannot. A predicate such as "is a solution" does not manufacture an algorithm. Conversely, a noncomputable mathematical object is entirely legitimate when that is what the author requested; the interface should label the choice rather than treating it as a defect.

7.7 Replay and source edits

Three receipts are useful but not interchangeable: a proof for the candidate expression, successful elaboration in the original context, and successful replay of the proposed source replacement. A printed suggestion may elaborate differently after local notation, implicit arguments, or instance search are reapplied. The final source edit should therefore be checked in a fresh copy of the relevant environment before being offered as a verified replacement.

A request for several synthesized subterms sharing metavariables is processed transactionally. Each candidate branch receives an isolated assignment state. Successful local assignments are committed only when the whole required result checks; otherwise a failed candidate could constrain later candidates through invisible state. The same discipline applies to speculative property rows.

Ranking should distinguish proof cost from exposition cost. A one-line appeal to a very general theorem may have large elaboration cost and weak explanatory value; a two-line direct argument may be easier to maintain. These are empirical ranking objectives, not logical conditions. No ranking policy may convert "retained after filtering" into "proves the requested behavior."

8 Certified computation becomes a source of refinements

8.1 One boundary, several mathematical relationships

Moraine uses the computation boundary established in round two, rather than creating a separate CAS type theory. A request refers to an anchored object a and a predicate $R(a, b)$ describing the permitted result b. A successful computation supplies b and a proof of $R(a, b)$. That proof may then introduce a refinement about a, about b, or about their relationship [1, 2].

For example, computing a polynomial factorization may first establish only

$$p = c \prod_i f_i^{e_i}.$$

Irreducibility and normalization are additional properties. Computing a jet establishes agreement below a specified order, not equality of complete power series. An interval encloses an exact real object; it is not a replacement interpretation of that object.

The typing layer is useful because consumers can state the relationship they need. Differentiating a jet consumes a precision-transfer theorem. Extracting a strict sign consumes an enclosure that excludes zero. Replacing an expression under an integral consumes an appropriate equality and domain theorem. An unsupported promotion becomes an unsatisfied typed obligation.

8.2 Guard inference must not weaken the request

Consider the familiar cancellation

$$\frac{x^2 - 1}{x - 1} = x + 1. \tag{18}$$

Over a field, the usual cancellation proof requires $x \neq 1$. A property-aware normalizer may infer this guard and ask for its proof. It may not silently replace the author's domain D by $D \setminus \{1\}$ and then declare the original request complete.

This distinction is particularly important with total operations. In Lean's usual field interpretation, division by zero is totalized. At $x = 1$, the left side of (18) is 0 and the right side is 2 over $\mathbb{R}$. Under an explicitly partial rational-expression interpretation, the left side is undefined there. The two requests have different denotations, even before any simplification. The surface must fix that interpretation; a guard does not erase it.

An obligation graph must also exclude circular justifications. A cancellation result conditional on $d \neq 0$ cannot itself be used to derive $d \neq 0$ unless a separate theorem establishes a valid noncircular argument. The final Lean proof term is the logical protection, while acyclic scoped dependency tracking keeps the user-facing account honest and avoids pointless search loops.

8.3 Quantitative rules are ordinary theorems

For $\mathsf{Close}_\varepsilon(a,b) := \|a-b\| \le \varepsilon$,

$$\mathsf{Close}_\varepsilon(a,b) \wedge \mathsf{Close}_\delta(b,c) \longrightarrow \mathsf{Close}_{\varepsilon+\delta}(a,c) \tag{19}$$

follows from the triangle inequality. If F is L-Lipschitz with $L \ge 0$, then

$$\mathsf{Close}_\varepsilon(a,b) \longrightarrow \mathsf{Close}_{L\varepsilon}(F(a),F(b)). \tag{20}$$

These laws permit backward demand propagation. A consumer asking for error at most η may request an input enclosure or approximation small enough that the proved transformation yields that bound. The planner chooses the requested precision; the theorem determines whether it suffices.

The same architecture supports formal-series congruence modulo X^N, but its rules are not numerical error rules. Differentiation generally lowers the known order by one. Inversion requires a unit constant coefficient. A series jet and a numerical interval do not share a generic arithmetic "confidence" level. Packages provide their own relations and implication theorems. There is no need to design a universal lattice of all mathematical effects before implementing these few interfaces.

8.4 The reflective execution bridge remains mandatory

Suppose a checker c has a proved soundness theorem

$$\forall q\, z,\ \ c(q,z) = \mathsf{true} \to \mathsf{Valid}(q,z). \tag{21}$$

An external execution that prints `true` has not supplied a Lean proof of the premise in (21). A strict path must reduce the actual checker application or construct an independently checked proof/certificate tying the input and output to that application. Reification must also connect the mathematical source goal to the checker's input representation. Verifying the algorithm but omitting this bridge does not certify a particular run [1, 2, 13].

The current Lean reference distinguishes proof-producing evaluation from native evaluation admitted through an axiom. Its validation discussion also records the change in native-evaluation axiom handling since Lean 4.29. A policy must inspect transitive axioms in the actual pinned environment, not blacklist a single historical constant name [13, 14]. The new typing layer cannot make a stronger promise than the evidence it consumes.

9 How far should Lean's type system actually change?

9.1 Four implementation levels

The phrase "extend the type system" covers several substantially different projects. They should be compared before selecting one.

The recommended starting point is the second level, implemented incrementally on top of the first. This is a genuine change to the source-language typing rules: refinement obligations participate in application, binding, and overload resolution. It is not a change to the consistency strength of the backend.

Table 2: Candidate levels of extension.

Level	What changes	Main question
Library only	Structures, theorems, attributes, tactics	Are better packages and `fun_prop`/`grind` integration already enough?
Refinement elaboration	Source judgments, binder forms, evidence indexing, implicit bundle adapters	Does a stable property-aware interface remove recurring user work without hiding meaning?
Explicit kernel refinements	A primitive checked view/package with evidence	Is there a measured checking or implementation benefit over a subtype encoding?
Semantic conversion extensions	Contextual equality reflection or arbitrary property entailment during conversion	Can the expanded equality/checking problem be controlled and justified?

9.2 An explicit primitive refinement could be conservative

A possible kernel primitive would have introduction and elimination forms

$$\frac{t : A \quad p : P(t)}{\mathsf{refine}(t,p) : \mathsf{Ref}(A,P)}, \qquad \frac{v : \mathsf{Ref}(A,P)}{\mathsf{forget}(v) : A},$$

$$\mathsf{evidence}(v) : P(\mathsf{forget}(v)), \qquad \mathsf{forget}(\mathsf{refine}(t,p)) \rightsquigarrow t. \tag{22}$$

With these explicit forms, the evident translation is to `Subtype` (or the `Certified` record) and its projections. The primitive is therefore a candidate representation convenience, not a new proof principle. An actual extension would still need a preservation argument for all its formation, conversion, universe, and reduction rules.

It could be worthwhile if profiling showed a large cost in repeated wrapper construction, projection normalization, or elaborator bookkeeping that a primitive checked view reliably removes. But proofs in `Prop` are already erased from compiled code [11]. Consequently, "proof erasure makes programs faster" is not an adequate argument for introducing (22). The relevant measurement is elaboration, kernel checking, or representational simplicity.

A more radical rule allowing a bare t to have type $\mathsf{Ref}(A,P)$ without any evidence expression would shift the burden into reconstructing a proof of $P(t)$ during checking. If the checker accepts only an explicit derivation certificate, the evidence has not really disappeared; it has moved. If it accepts an arbitrary solver answer, the logical boundary has changed. The design must choose and document which situation it intends.

9.3 Do not make arbitrary theorems into definitional equality

It is tempting to decree that logically equivalent refinements are identical for conversion, or that a proof of $t = u$ lets the kernel regard t and u as definitionally equal everywhere. This would remove some visible transports, but it also changes the checker from a controlled computational conversion test into a substantially broader context-dependent reasoning procedure.

For arbitrary refinement predicates, deciding semantic inclusion already contains arbitrary theorem entailment: take the source predicate to be `True` and the target predicate to be an arbitrary constant

proposition. No complete terminating inclusion algorithm is promised here. Adding that task to conversion would make routine type checking depend on the same open-ended search that the architecture deliberately keeps outside the kernel.

A proof-directed elaborator can obtain most of the desired convenience by inserting explicit transports and implication applications. Lean distinguishes these propositional operations from definitional equality [12]. The user need not see every transport, but a compact checker can still see its proof. This is a stronger practical argument than a blanket prohibition on kernel experiments: it identifies which convenience can be delivered without expanding conversion.

9.4 Dependent indices and quotients remain real boundaries

Suppose a vector has length $n+m$ and a consumer expects length $m+n$. The elaborator may prove $n+m=m+n$ and insert a dependent transport. It must not erase the length index, reinterpret the vector's storage, or declare all arithmetic expressions interchangeable. The transport is part of the generated core term.

Similarly, a function on representatives of a quotient descends only after a respectfulness proof. A refinement saying that the representative function is continuous is not that proof. A language that makes quotient reasoning natural should generate and solve the respectfulness obligation, not delete it from the type discipline.

Equivalence of carriers is also not equality of the chosen algebraic structure. If the source wants transport along an equivalence, the map, its laws, and the transported operations must be specified. Replacing this problem by a different foundation is a separate research program, not a small extension of Lean's existing proof-irrelevant equality interface.

9.5 A gate for future kernel work

A kernel experiment should begin only after the elaborator has accumulated representative proof traces. The gate is concrete: identify repeated core terms whose cost dominates a benchmark; propose one primitive or conversion rule; state its exact translation or semantics; implement a checker prototype; and compare against optimized ordinary Lean on the same exported theorems.

The first extension to test, if any, would be an explicit checked refinement view with a conservative translation, not implicit arbitrary entailment. If it gives no material advantage over improved elaboration and proof sharing, it should be removed. The mathematical language should not acquire a permanent trusted feature merely because it made an early parser easier to write.

10 An executable slice: universal affine length contracts

10.1 Why this fragment

The round-two criticism of arithmetic-only companions is well taken: a useful experiment should exercise a semantic boundary, not just reproduce numbers from a worked example [2]. The companion here therefore checks a specific behavioral judgment attached to an exact candidate. Its domain is deliberately small enough to state its soundness and completeness precisely.

Fix k input lists of natural numbers. Expressions are built from input variables x_i, the empty list, cons with a literal natural-number head, append, reverse, and elementwise successor. No arbitrary

provider, filter, recursion scheme, custom list encoding, or effectful computation is admitted. All expressions denote total list functions in the usual recursive way.

For each expression e, construct a nonnegative affine form

$$L_e(n_0, \ldots, n_{k-1}) = c_e + \sum_{i<k} a_{e,i} n_i, \quad c_e, a_{e,i} \in \mathbb{N}, \tag{23}$$

using these rules:

$$\begin{aligned} L_{x_i}(n) &= n_i, \\ L_{[]}(n) &= 0, \\ L_{b::e}(n) &= 1 + L_e(n), \\ L_{e_1 ++ e_2}(n) &= L_{e_1}(n) + L_{e_2}(n), \\ L_{\mathsf{reverse}(e)}(n) &= L_e(n), \\ L_{\mathsf{mapSucc}(e)}(n) &= L_e(n). \end{aligned} \tag{24}$$

The implementation represents (23) by a constant and an exact-length coefficient vector. It checks arity before combining vectors, avoiding silent truncation.

Theorem 10.1 (Length-model soundness)**.** *For every expression e in this grammar and every environment of lists $\mathbf{x} = (x_0, \ldots, x_{k-1})$,*

$$|[\![e]\!](\mathbf{x})| = L_e(|x_0|, \ldots, |x_{k-1}|). \tag{25}$$

Proof. Induct on the expression. An input variable yields the corresponding input length; the empty list has length zero. Cons adds one to length. Append adds the two child lengths, to which the induction hypotheses apply. Reversal and mapping preserve length. Each step is exactly the corresponding equation in (24), proving the claim. □

Theorem 10.2 (Complete affine coefficient criterion)**.** *Let $L(n) = c + \sum_i a_i n_i$ and $M(n) = d + \sum_i b_i n_i$ have natural-number coefficients and the same arity. Then $L(n) = M(n)$ for every $n \in \mathbb{N}^k$ if and only if $c = d$ and $a_i = b_i$ for every i.*

Proof. Equality of all coefficients gives equality of values by substitution. In the other direction, evaluation at the zero vector gives $c = d$. Evaluation at the ith unit vector gives $c + a_i = d + b_i$; cancellation of the common natural-number summand yields $a_i = b_i$. For $k = 0$, the zero-vector argument is the entire proof. □

Corollary 10.3 (Complete behavioral check within the grammar)**.** *An expression e satisfies the universal length specification $|[\![e]\!](\mathbf{x})| = M(|x_0|, \ldots, |x_{k-1}|)$ exactly when its computed affine coefficients equal those of M.*

Proof. Sufficiency follows from Theorems 10.1 and 10.2. For necessity, every vector in $\mathbb{N}^k$ is realized by lists of those lengths, for example lists consisting entirely of zeros. The assumed universal list law and (25) therefore give equality on all length vectors, to which Theorem 10.2 applies. □

This gives an exact refutation procedure as well. If constants differ, use input lengths all zero. Otherwise choose a unit vector at a differing coefficient. These are counterexamples derived from a complete model, not a random testing heuristic. Finite observations *can* support a universal conclusion when a proved finite-characterization theorem applies. Without the affine-shape premise, the same observations say much less: an arbitrary function could agree on all those inputs and behave differently on a longer list.

10.2 What the executable checker accepts

The Python request contains an epoch label, arity, exact source expression, and desired affine specification. A certificate contains the claimed normal form and the same source information. Verification reconstructs the length law from the expression, validates arities and resource bounds, checks the original expression and epoch, and compares the reconstructed coefficients with both the certificate and the requested specification.

Checking exact source identity is a protocol condition in addition to the length theorem. A proof about reversal cannot be replayed as evidence about an already anchored identity function merely because their normal forms agree. The target could be certified independently, but that is a new justified operation, not receipt reuse by relabeling.

The mathematical coefficient criterion is unbounded for finite expressions. The executable version has explicit engineering limits: at most 64 inputs, depth 64, 10,000 visited syntax nodes, and 4,096 bits per admitted natural number. Exceeding a limit is an unsupported request, not a refutation of a behavioral law. Coefficient comparison costs $O(k)$ arithmetic comparisons after structural normalization; normalization uses $O(k)$ arithmetic work per nontrivial syntax node. This counts operations, not constant-time arithmetic on arbitrarily large integers.

10.3 What was actually run

The archive includes `length_contracts.py`, `test_length_contracts.py`, and the captured test output. The run completed all 32 named tests. Seven positive tests cover the basic constructors and input arities. Negative and boundary tests cover corrupt coefficients, false requested laws, a different source with the same length behavior, epoch mismatch, coefficient-vector truncation, unsupported syntax, invalid input indices, non-natural coefficients, and resource limits. Other tests exercise the complete counterexample construction and the distinction between length preservation and identity of list functions.

One regression test generates 400 expressions using a fixed random seed and compares the concrete list interpreter with the affine model at all $4^3 = 64$ input-length vectors in $\{0, 1, 2, 3\}^3$, for 25,600 comparisons. Another checks the zero/unit-vector counterexample construction for all 729 ordered pairs among 27 small two-input affine forms. These are executable regression checks, not the proofs of Theorems 10.1 and 10.2.

10.4 What this does not establish

The Python checker is not Lean-verified, not a secure process boundary, and not an implementation of general refinement typing. Its epoch is an explicit string used to test the matching rule; it does not reproduce Leant's generative parametricity guarantee. Its receipt is a reporting object, not an unforgeable logical proof. In particular, the tests do not validate neighborhood scopes, naturality inference, the proposed syntax, or the full source reconstructions.

The ordinary-Lean companion states the core contract combinators and the denotational half of length soundness, without `sorry` or new axioms. It was not compiler-tested here. It does not yet connect the coefficient-vector Python normalizer to a Lean reflection theorem. Consequently, this article claims a mathematical proof of the specified model and an executed reference implementation, *not* a kernel-verified universal-contract service.

The next implementation step is narrowly defined: port the affine coefficient representation and

normalization to Lean, prove its denotation agrees with the structural model, and expose the resulting checker behind the existing Leant request boundary. This is more informative than adding another dozen property keywords before the first property has a complete proof-producing path.

11 Implementation architecture and a discriminating evaluation

11.1 A small elaborator, not a second proof assistant

The first implementation can be organized around six components. Their interfaces are more important than the final names of the source commands.

Component	Responsibility and retained invariant
Typed syntax expansion	Elaborate the mathematical carrier, binders, and property predicates before optional search. Preserve explicit data-hole permissions and the public theorem statement.
Evidence index	Associate checked proof expressions with exact object anchors and local contexts. Store scopes and dependencies, not Boolean truth labels.
Interface adapters	Supply theorem-backed conversions between unbundled functions with laws and existing bundled objects. Record projection equations and operation parameters.
Obligation scheduler	Dispatch property goals to local evidence, theorem composition, `fun_prop`, `grind`, Leant, or certified domain services under explicit work budgets.
Backend and policy checker	Check reconstructed expressions and property proofs in the pinned Lean environment; audit allowed transitive axioms and unresolved metavariables.
Source replay and explanation	Validate the exact proposed source replacement; retain a readable proof graph and distinguish reason-constrained proofs from hint-guided search.

The evidence index is an elaboration aid. Its authoritative content is the proof expression and its checked type. A corrupted index may waste work or produce a rejected term; it must not confer a new primitive proof rule. This is the same basic separation used by proof-producing tactics, whose output is checked by Lean rather than trusted as a theorem oracle [15].

11.2 A deterministic cheap tier and explicit expensive tiers

Property inference should have a cheap predictable tier before invoking general search. It begins with exact local evidence and projections from existing bundles, then applies a small set of registered implication and composition rules. Requests for continuity can use `fun_prop`; compatible algebraic or logical side conditions can use `grind`, which already maintains cooperating proof-producing engines [16, 17].

For inexpensive closure, admit a finite set of anchored terms and property schemas. Rules with fully instantiated conclusions over that finite set produce at most finitely many new property keys.

A worklist can therefore terminate without claiming completeness for all mathematics. A rule that creates new terms, recursively explores a definition, or starts arbitrary search belongs in a separately budgeted tier.

A candidate property key cannot be committed while its proof still depends on unassigned metavariables or on the very property being constructed. The scheduler may record a pending obligation, but consumers see it as pending. Failed speculative branches discard their evidence and metavariable assignments. This is ordinary transactional elaboration applied systematically to refinement inference.

11.3 Package contracts

A property package declares a Lean predicate, the argument that serves as its main object anchor, additional semantic parameters, and a set of proved rules. A bundled-interface adapter declares its target structure, constructor, projection equations, and law obligations. A modality package declares the proposition denoted by its scope notation and its conversion theorems.

For example, a `linear` property does not merely have the name "linear." It records the scalar structure, module structures, the underlying function, and the exact additive and scalar laws. A registered adapter to a `LinearMap` identifies its `toFun` with that same function. A later adapter to a `LinearEquiv` adds a particular inverse and both inverse laws. No adapter gets to choose new algebraic data while presenting the operation as a proof-only refinement.

Registration must be independently checkable. It is reasonable for the first version to require a human-written Lean theorem and a small declarative adapter, rather than infer arbitrary package semantics from theorem names. Automated package suggestions can be added later; acceptance still checks their terms.

11.4 Diagnostics should expose the missing mathematical distinction

The system should avoid ending a long search with only "failed to synthesize instance." More useful diagnostics refer to the typed obligation graph:

```
Cannot differentiate this equality at x.
Available: f(x) = g(x).
Required: f = g eventually near x.
A theorem giving equality on an open neighborhood would suffice.
```

```
The candidate inhabits List Nat -> List Nat.
Its proved length law is length(result) = 2 * length(arg0).
Requested: length(result) = length(arg0).
A counterexample has input length 1.
```

```
The equivalence family has been constructed.
The postcomposition naturality law remains unproved.
No natural-equivalence interface has been exported.
```

These messages are consequences of named proposition families and failed bridges, not an attempt to explain an opaque solver by inventing a story after it finishes.

11.5 Caching and collaborative edits

A cached proof may be reused when its environment and scope are compatible and its proposition is the one requested, possibly after an explicit weakening or transport. A proof under assumption H is not unconditional cache content; it must remain parameterized by H or be supplied with a current proof of H. Different axiom policies also affect admissibility even when the resulting mathematical proposition is the same.

For collaboration, the durable artifact should be source plus exported Lean proofs and a reconstructible manifest, not a mutable shared "known facts" store. Two authors can merge source changes and re-elaborate affected obligations. The merged environment must not simply union local proof rows, since their binders, object anchors, and assumptions may no longer agree. A semantic diff should highlight changed carriers, predicates, quantifier order, instance arguments, and axiom dependencies separately from changed proof routes. This is a concrete response to the collaboration gap noted by the second synthesis [2].

11.6 The experimental comparison must isolate the new layer

The four cases in this article are development cases. They must not later be reported as an untouched holdout set. Choose additional files only after freezing the initial adapters and make their role explicit in the benchmark manifest. A useful evaluation has four arms:

Table 4: Ablations that distinguish better libraries from a better language.

Arm	Available facilities
A	Current ordinary Lean, with an experienced author's idiomatic proof.
B	Ordinary Lean plus all new theorem packages, adapters, and computation providers developed for the experiment.
C	Arm B plus refinement-aware binding, scoped evidence indexing, and behavioral obligations; otherwise ordinary Lean proof syntax.
D	Arm C plus the controlled mathematical proof surface and its explanation contracts.

A versus B measures library engineering. B versus C measures the property-aware typing interface. C versus D measures the proposed proof language. The same provider budgets, theorem statements, instance choices, and axiom policies must be used when making the comparisons.

Measure author-supplied mathematical steps, representation annotations, unresolved obligations, completion rate under fixed budgets, cold discovery time, warm elaboration time, backend checking time, proof size, and repair effort after controlled library changes. Count package-writing effort separately and report its amortization over later uses. Source line count alone conflates deletion of mathematics with harmless elision of evidence.

The first substantive success criterion is not a target compression percentage. It is that C removes repeated interface obligations on previously unseen cases while preserving their exact statements and performing no worse on adversarial mutations. D must then improve readability or authoring effort over C. If B captures nearly all the benefit, the appropriate result is a Lean library and better tactics, not a new standalone language.

11.7 A proposed adversarial suite

The following are *proposed integration tests*, not tests already executed by the companion. Their purpose is to falsify the dangerous shortcuts the design might otherwise encourage.

Mutation	Required response
Point equality substituted for neighborhood equality	The derivative-transfer obligation fails; $f(t) = t$, $g(t) = 0$ at zero supplies a concrete separating case.
Induction hypothesis specialized to one point	The successor proof cannot advertise an on-set or neighborhood equality without an additional argument.
Endpoint deletion labeled as equality	The singleton/empty-set example is rejected, while the norm-error contract is accepted.
Negative integer indices reindexed through `toNat`	Bijection obligations fail for the $\{-1, 0\}$ collision.
Inverse law omitted or anchored to another map	No equivalence bundle is exported until the law is proved about the selected maps.
Arbitrary objectwise equivalences labeled natural	The explicit naturality obligation must be proved; well-typed components are insufficient.
Different action or scalar structure on the same carrier	Old linearity or equivariance evidence cannot be reused without a theorem relating the structures.
Evidence from a sibling branch	Scope matching refuses reuse; a valid case eliminator may still prove a common consequence.
A guard inferred from its own conditional result	No cyclic proof evidence is committed.
Behavioral candidate merely retained by a filter	No universal-property row is introduced without a separate proof-producing route.
Type-correct candidate changes a fixed source object	The original-target check refuses the replacement unless the source authorized data synthesis.
Native evaluation violates the permitted axiom profile	The proof is reported as inadmissible under that profile, even if it checks with additional axioms.

Omitting a premise does not always make a conclusion false: another valid proof may establish it. Tests must therefore include concrete separating instances, and distinguish a failed named method from failure of the mathematical theorem. A reason-constrained step should not be marked as that method merely because unrelated automation proves the final result.

12 Positioning and remaining research questions

12.1 What is established, and what is a new design claim

Refinement types and automatic inference from a bounded predicate vocabulary are established ideas. Liquid Types combines Hindley–Milner inference with predicate abstraction [18]. F* distinguishes refinements of values from specifications of computations [19]. These precedents are directly relevant; Moraine does not claim to invent refinements, pre/postconditions, or behavioral verification.

Within Lean, ordinary propositions, bundled maps, subtypes, proof-producing tactics, and function-property inference already supply much of the semantic machinery. The distinguishing proposal is their integration into an *anchor-preserving mathematical typing interface*: a stable carrier term, scoped proof-bearing properties, relational specifications that remain attached to all their parameters, and a disciplined transition between unbundled local reasoning and bundled library APIs.

Relative to round two, the main change is that the request-and-evidence protocol is no longer only the boundary of a CAS command. It participates in binding, function application, composition, implicit interface selection, and synthesis of mathematical data. The architecture still has to prove its value against the same capabilities exposed directly in Lean.

12.2 Which inferred properties should be visible?

Showing every derived fact would be as unhelpful as showing every generated transport. Hiding all of them would make a failed proof mysterious. One possible policy is to display user-declared properties and properties consumed by the current step, while keeping routine consequences available through expansion. The expanded view must show the actual predicate, domain, proof dependencies, and derivation route.

A further question is whether a property should be part of a stable public interface or a local inference result. Public interfaces should generally state the guarantees on which clients may rely. Incidental inferred properties can be used locally but should not silently become compatibility promises of a published package.

12.3 Can one recover mathematical structure from existing proofs?

A promising migration tool could recognize recurring proof patterns: a function with additive and scalar laws, a bijective reindexing proof, or a neighborhood conversion preceding derivative congruence. It could propose a property-bearing interface and a shorter source rendering while retaining the original proof as a certificate.

The original proof supplies a valuable oracle for target preservation, but does not settle exposition. For example, replacing a direct proof by an existing large theorem may shorten text without representing the author's reasoning. Such refactorings should be suggestions with a semantic diff, not automatic changes to the asserted explanation.

12.4 Can evidence summaries be made cheap enough?

A large mathematical development may have many relevant properties of many terms. Unrestricted forward saturation is not a realistic default. Demand-driven inference, finite cheap closure, proof sharing, and context-local caching are therefore essential engineering hypotheses. They are not guaranteed performance wins until measured on larger developments.

The hard cases are likely to include dependent indices with many transports, large algebraic instance arguments, and broad relational properties such as naturality. These should be evaluated early rather than extrapolating from simple scalar continuity or affine length examples.

12.5 The first vertical slice

The initial implementation should accept a small set of `fix/know/use` refinement forms, generate ordinary Lean proof obligations, and support one well-defined bundle adapter. Evaluation and constant extension in the coinduced-invariants theorem are a suitable test because the mathematical maps are simple but their admissibility and inverse laws are not merely carrier information.

In parallel, the affine length fragment has a sharply specified route to a proof-producing Leant integration: formal denotation, coefficient normalization, soundness, complete coefficient comparison, and exact-target replay. These two slices test different requirements: mathematical interface construction and behavioral synthesis. Only after they work should the grammar expand to neighborhood blocks and natural representing families.

13 Conclusion

The best reason to extend a mathematical type system is not to place every true theorem into conversion. It is to let proved properties become dependable interfaces at the points where mathematical reasoning consumes them.

Moraine therefore keeps the object fixed and lets its classification grow. An ordinary function can acquire continuity, linearity, inverse, or naturality interfaces without being silently replaced. A local equality retains its set or filter. An approximation retains its error or precision. A synthesized program acquires a behavioral refinement only when the exact universal statement has been established about that program.

The type-theoretic proposal is conservative but not merely cosmetic: refinement rules govern source binding, application, composition, and admissible adapters. They translate to explicit Lean proofs. More ambitious kernel primitives remain possible, but their value must be demonstrated against that translation rather than assumed. The resulting research program is small enough to begin with real proof obligations, and precise enough that a failed experiment would still yield useful theorem packages, checked adapters, and better diagnostics.

A Reference semantics and artifact status

The archive contains the following files.

File	Purpose
`moraine.tex`, `moraine.pdf`	Article source and rendered article.
`README.md`, `build.sh`	Reproduction instructions and scope of validation.
`sources.json`	Full repository pins, source paths, and documentation URLs.
`companion/Contracts.lean`	Ordinary Lean reference encodings; not compiler-tested here.
`companion/length_contracts.py`	Executed reference checker for the fixed grammar.
`companion/test_length_contracts.py`	Reproducible regression tests.
`companion/test-results.txt`	Captured output of the 32-test run.

The Lean reference distinguishes a fixed value with evidence from a function contract. It includes

the following logical interfaces (abbreviated here in ASCII notation):

```
Refines P Q := forall x, P x -> Q x
MapSpec P Q f := forall x, P x -> Q x (f x)
RelSpec R S f := forall x y, R x y -> S (f x) (f y)
InversePair f g :=
  (forall x, g (f x) = x) /\ (forall y, f (g y) = y)
```

The actual `.lean` file uses Lean's ordinary Unicode logical notation and provides elementary proof terms for weakening, composition, and inverse-pair consequences. Its list-expression model expresses the recursive denotation and structural length equations used in Theorem 10.1. No claim in this article depends on assuming that an unrun Lean file compiled successfully.

The reported test run was executed with the Python standard library. The article's soundness arguments are mathematical proofs of the stated models; the Python tests are evidence about the implementation, and the Lean file is a reference encoding awaiting compiler validation. These three statuses should remain separate in future iterations as well.

B Translation patterns to preserve during implementation

A locally refined object

The source binder `fix x : A where P(x)` introduces $x : A$ and $h_x : P(x)$. A later theorem using only x receives that same term. A theorem requiring $P(x)$ receives h_x or a derived proof. Returning both as a first-class value constructs a subtype-like package; no global coercion changes x's identity.

A refined total function

The source `fix f : A -> B where require P(x) => Q(x,f(x))` introduces a total $f : A \to B$ and a proof of (1). Application at a always denotes $f(a)$; claiming its specified postcondition additionally consumes a proof of $P(a)$. It does not turn every call into existential search for a suitable output.

A restricted-domain function

The source input type `(x : A where P(x)) -> B` is a different form. Its first-class domain is a refined package, or an explicitly curried value/proof binder. The implementation cannot erase the precondition and expose an arbitrary total function on A unless it also constructs an extension.

Export from a local scope

A proof of Q under $h : H$ exports as $H \to Q$ when Q does not itself require additional local data. If local data occur, they remain explicit parameters or are quantified with a justified introduction rule. The implementation computes dependencies from terms; it does not decide that an assumption was irrelevant because a human explanation omitted its name.

Evidence from a computation

A certificate checker theorem concludes $R(a, b)$ about the anchored request a and computed value b. The refinement index may store that proof. Promoting it to a different predicate, source object, domain, or precision is another theorem application. A valid result in a different request context is not accepted by copying the original receipt label.

References

[1] Brainstorm round-two synthesis. *Mathematical Objects, Certified Computation: A Unified Evaluation of Nine Proof-Language Proposals*. 6 September 2026. https://github.com/VladimirReshetnikov/Brainstorm/blob/b052ec011beb96a7c48df2a9988823f0f779b445/docs/round-2/synthesis/unified-report.tex.

[2] Brainstorm round-two unified report. *Nine Workbenches: Certified Computation in a Mathematician's Proof Language over Lean*. 6 September 2026. https://github.com/VladimirReshetnikov/Brainstorm/blob/b052ec011beb96a7c48df2a9988823f0f779b445/docs/round-2/unified_report/unified_report.tex.

[3] Vladimir Reshetnikov, ProveIt contributors. `AutonomousIteratedDeriv.lean`, especially `AutonomousODE.iteratedDeriv_eq_comp`. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean.

[4] Vladimir Reshetnikov, ProveIt contributors. `GKBProcessAux.lean`, lines 1–230 of the inspected snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/NumberTheory/IntegerPoints/Lean/IntegerPoints/GKBProcessAux.lean.

[5] Contributors to `anthropics/fermats-last-theorem`. `S_Rep_nonempty_invariants_coind_equiv.lean`. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_Rep_nonempty_invariants_coind_equiv.lean.

[6] Contributors to `anthropics/fermats-last-theorem`. `S_Algebra_exists_algHom_equiv_pi.lean`. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_Algebra_exists_algHom_equiv_pi.lean.

[7] Leant contributors. *Host-native behavioral constraints in both REPLs*. 20 August 2026. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/docs/reports/2026-08-20-host-native-behavioral-constraints.md.

[8] Leant contributors. `src/Leant/Synth/Verification.hs`. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Verification.hs.

[9] Leant contributors. `src/Leant/Synth/BehavioralSelection/Internal.hs`, especially the verified occurrence and generative epoch interfaces. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/BehavioralSelection/Internal.hs.

[10] The Lean Language Reference. *The Type System*. Consulted 6 September 2026. https://lean-lang.org/doc/reference/latest/The-Type-System/.

[11] The Lean Language Reference. *Propositions*. Consulted 6 September 2026. https://lean-lang.org/doc/reference/latest/The-Type-System/Propositions/.

[12] The Lean Language Reference. *Propositional Equality*. Consulted 6 September 2026. https://lean-lang.org/doc/reference/latest/Basic-Propositions/Propositional-Equality/.

[13] The Lean Language Reference. *Validating a Lean Proof*. Consulted 6 September 2026; especially native-evaluation assumptions and validation of the intended statement. https://lean-lang.org/doc/reference/latest/ValidatingProofs/.

[14] The Lean Language Reference. *Tactic Reference*, sections on decision procedures and call-by-value evaluation. Consulted 6 September 2026. https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/.

[15] The Lean Language Reference. *Tactic Proofs*. Consulted 6 September 2026. https://lean-lang.org/doc/reference/latest/Tactic-Proofs/.

[16] mathlib documentation. *Mathlib.Tactic.FunProp*. Consulted 6 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/FunProp.html.

[17] The Lean Language Reference. *The grind tactic*. Consulted 6 September 2026. https://lean-lang.org/doc/reference/latest/The--grind--tactic/.

[18] Patrick Rondon, Ming Kawaguchi, Ranjit Jhala. *Liquid Types*. PLDI, 2008. Authors' paper page and abstract. https://goto.ucsd.edu/~rjhala/papers/liquid_types.html.

[19] F* tutorial. *Primitive Effect Refinements*, in *Proof-Oriented Programming in F**. Consulted 6 September 2026. https://fstar-lang.org/tutorial/book/part4/part4_pure.html.